Database Users, Roles, Privileges, and Access Control

Group 10 (MESA - Order and Reserve)

Section	Contents
Cover and links	**GROUP 12: Amoranto, Denielle; Eurescial, Eunice; Francisco, Helix** **Project Assigned:** MESA - Order and Reserve **Repository Link:** **https://github.com/kathlenmaecallos11-arch/Restaurant-Online-Ordering-Table-Reservation-System** **Our Group Repository:**
Projects set up	**The Restaurant Online Ordering and Table Reservation System repository was used as the basis for implementing the database access-control plan. The project is a restaurant management application designed to support online food ordering and table reservation operations.** **The first step was to examine the existing project files and understand how the application manages restaurant information, customer records, orders, and table reservations. The repository structure and available database-related files were reviewed to identify the database configuration and determine the database objects that would require access restrictions.** **The database used for this activity was PostgreSQL. The default schema considered for the access-control implementation was the public schema. The existing database structure was intended to be retained, and the access-control implementation was prepared separately to avoid modifying the original system functionality.** **The database was inspected using PostgreSQL queries to identify the existing database name, schemas, tables, columns, sequences, and database roles. This information served as the basis for preparing the role assignments and privilege configuration.** **After reviewing the database setup and access-control requirements, a separate folder named Activity-4 was prepared to organize the SQL scripts and documentation.** **The following files were created:** **00_inspect_database.sql**

	01_create_roles.sql **02_grant_privileges.sql** **03_revoke_privileges.sql** **04_test_privileges.sql** **access_control_plan.md** **screenshots/** **These files were organized to make the implementation easier to understand, execute, verify, and maintain.**
Access-control plan	**The access-control plan was created to determine which database operations should be allowed for each user role within the Restaurant Online Ordering and Table Reservation System.** **The implementation follows the Principle of Least Privilege, meaning that each database role should only receive the permissions necessary for its assigned responsibilities.** **Five PostgreSQL group roles were established to represent the different responsibilities within the restaurant system.** **Administrator Role (restaurant_admin)** **The administrator role is responsible for managing and maintaining the Restaurant Online Ordering and Table Reservation System database.** **Its responsibilities include viewing database records, inserting new records, updating existing information, deleting records when necessary, and managing database access permissions.** **The administrator role receives ALL PRIVILEGES on all existing tables and sequences within the public schema.** **This role is intended only for authorized database administrators and should not be used for ordinary application operations.** **Manager Role (restaurant_manager)** **The manager role is intended for personnel responsible for overseeing restaurant operations, customer orders, and table reservations.**

	Its responsibilities include viewing restaurant records, monitoring customer orders, reviewing table reservations, and accessing relevant restaurant information. **The manager role initially receives SELECT privileges on all existing tables within the public schema.** **This allows managers to view available restaurant information without automatically receiving permission to modify or delete database records.** **Additional INSERT and UPDATE privileges may be assigned to specific tables when required by actual management responsibilities.** **Staff Role (restaurant_staff)** **The staff role is intended for restaurant employees responsible for handling customer orders and table reservations.** **Its responsibilities include viewing customer orders, checking reservation information, monitoring restaurant transactions, and accessing information needed for daily restaurant operations.** **The staff role initially receives SELECT privileges on all existing tables within the public schema.** **The role is restricted from performing DELETE operations through the prepared REVOKE statement. Other modification privileges should only be granted when required by the actual application workflow.** **This helps prevent unauthorized removal of restaurant records.** **Customer Role (restaurant_customer)** **The customer role is intended for users who interact with the restaurant system to access customer-related information and services.** **Its responsibilities include accessing permitted customer information, viewing relevant restaurant records, and using customer-facing system features.** **The customer role initially receives SELECT privileges on all existing tables within the public schema.** **It is restricted from performing INSERT, UPDATE, and DELETE operations through the prepared REVOKE statements.**

In an actual deployment, customers should access their own records and perform ordering and reservation transactions through the application rather than receiving unrestricted direct database access.

Read-Only Role (restaurant_readonly)

The read-only role is intended for authorized users who only need to view database information.

Its responsibilities include viewing restaurant records, inspecting available database information, and reviewing records for monitoring and reporting purposes.

The read-only role receives SELECT privileges on all existing tables within the public schema.

It cannot insert, update, delete, or truncate records. It is also restricted from modifying database sequences.

This ensures that users assigned to this role can inspect database information without making unauthorized modifications.

Access-Control Matrix

The following matrix summarizes the intended database permissions assigned to each group role.

Database Operations	Administrator	Manager	Staff	Customer	Read-Only
SELECT	Allowed	Allowed	Allowed	Allowed*	Allowed
INSERT	Allowed	Not granted	Not granted	Denied	Denied
UPDATE	Allowed	Not granted	Not granted	Denied	Denied

DELETE	**Allowed**	**Not granted**	**Denied**	**Denied**	**Denied**
TRUNCATE	**Allowed**	**Not granted**	**Not granted**	**Not granted**	**Denied**
Sequence Privileges	**Allowed**	**Not granted**	**Not granted**	**Not granted**	**Denied**
CREATE in public schema	**Not explicitly granted**	**Not explicitly granted**	**Not explicitly granted**	**Not explicitly granted**	**Not explicitly granted**

***Customer SELECT access is based on the initial database grants. Actual customer access should be restricted to appropriate customer-related records through the application.**

The permissions listed above represent the intended access-control configuration based on the prepared SQL scripts. Actual privileges must be verified against the implemented PostgreSQL database permissions.

The roles should also be reviewed for inherited privileges because PostgreSQL permissions can be obtained through other role memberships and grants.

Database Users and Role Assignments

The following PostgreSQL database login accounts were prepared:

Database User	Assigned Role
admin_user	**restaurant_admi**

	n **manager_user** **restaurant_manager** **staff_user** **restaurant_staff** **customer_user** **restaurant_customer** **viewer_user** **restaurant_readonly** **The five group roles were created using the NOLOGIN attribute. This means they function as permission groups and cannot be used as direct login accounts.** **The login accounts were created using the LOGIN attribute and assigned to their corresponding group roles through PostgreSQL GRANT statements.** **This allows database users to inherit the permissions assigned to their respective roles.** **The administrator account is intended for database administration, the manager account is intended for overseeing restaurant operations, the staff account is intended for handling restaurant activities, the customer account is intended for customer-related access, and the viewer account is intended for read-only access.**
SQL scripts summary	**Five SQL scripts were prepared to inspect, implement, and verify the database access-control configuration.** **File 1: 00_inspect_database.sql** **The first SQL script was used to inspect the existing PostgreSQL database structure before implementing the access-control**

	configuration. **The script contains queries that display the current database name, available database schemas, existing tables, table columns, sequences, and views.** **The inspection focused on identifying:** ● **Existing database schemas.** ● **Existing tables in the public schema.** ● **Table column names and data types.** ● **Existing database sequences.** ● **Existing database views.** ● **Current PostgreSQL roles.** **This step was important because the access-control implementation needed to be based on the actual database structure instead of using placeholder table names.** **The inspection also helped identify the database objects that would receive the assigned privileges and helped prevent errors caused by referencing tables that do not exist.** **File 2: 01_create_roles.sql** **The second SQL script was used to create the five PostgreSQL group roles and their corresponding login accounts.** **The following group roles were created:** **restaurant_admin** **restaurant_manager** **restaurant_staff** **restaurant_customer** **restaurant_readonly** **The following database login accounts were also created:** **admin_user**

	manager_user **staff_user** **customer_user** **viewer_user** **Each login account was assigned to its corresponding group role using PostgreSQL GRANT statements.** **The group roles were created using the NOLOGIN attribute, while the database login accounts were created using the LOGIN attribute.** **This approach allows permissions to be managed through group roles rather than assigning privileges individually to every database account.** **The administrator account is intended for database administration, the manager account is intended for restaurant management, the staff account is intended for operational activities, the customer account is intended for customer-related access, and the viewer account is intended for read-only access.** **File 3: 02_grant_privileges.sql** **The third SQL script was used to assign the required database permissions to each group role.** **The implementation uses the public schema as the target schema for the initial privilege configuration.** **All five group roles were given USAGE permission on the public schema so they could access database objects for which they have the appropriate privileges.** **The administrator role received ALL PRIVILEGES on all existing tables and sequences in the public schema.** **The manager role received SELECT privileges on all existing tables in the public schema.** **The staff role received SELECT privileges on all existing tables in**

	the public schema. **The customer role received SELECT privileges on all existing tables in the public schema.** **The read-only role received SELECT privileges on all existing tables in the public schema.** **The use of ON ALL TABLES IN SCHEMA public allows PostgreSQL to assign permissions to all existing tables without manually specifying each table name.** **Similarly, ON ALL SEQUENCES IN SCHEMA public allows privileges to be assigned to all existing sequences in the specified schema.** **These statements make privilege assignment easier when working with a database containing multiple tables and sequences.** **The initial configuration provides a basic permission structure. Additional table-specific privileges may be granted to managers and staff depending on the actual requirements of the restaurant application.** **File 4: 03_revoke_privileges.sql** **The fourth SQL script was used to remove unnecessary privileges from restricted roles.** **The staff role was restricted from performing DELETE operations on existing database tables.** **The customer role was restricted from performing INSERT, UPDATE, and DELETE operations.** **The read-only role was restricted from performing INSERT, UPDATE, DELETE, and TRUNCATE operations.** **The read-only role was also restricted from using or updating existing database sequences.** **These restrictions help prevent ordinary users from performing unnecessary database operations or making unauthorized modifications.**

	The administrator role retains its assigned table and sequence privileges, while the manager, staff, customer, and read-only roles are limited according to their initial responsibilities. **This helps implement the Principle of Least Privilege by preventing unnecessary access rights.** **It is important to note that PostgreSQL REVOKE statements remove specific grants but do not automatically cancel permissions inherited from other roles or separate grants.** **File 5: 04_test_privileges.sql** **The fifth SQL script was used to check whether the assigned database permissions were properly applied.** **The script contains queries that display the assigned database roles, verify role memberships, and inspect table privileges using PostgreSQL privilege-checking functions.** **The verification focused on checking whether:** ● **The administrator has SELECT privileges.** ● **The administrator has full table privileges.** ● **The administrator has sequence privileges.** ● **The manager role can view existing records.** ● **The staff role can view existing records.** ● **The staff role cannot delete records.** ● **The customer role can view existing records.** ● **The customer role cannot insert records.** ● **The customer role cannot update records.** ● **The customer role cannot delete records.** ● **The read-only role can view existing records.** ● **The read-only role cannot insert records.** ● **The read-only role cannot update records.** ● **The read-only role cannot delete records.** **These checks help confirm that each role follows the intended access-control configuration.** **The verification script uses PostgreSQL functions such as has_table_privilege() to determine whether specific database roles have permission to perform certain operations.**

<table>
<tr><td></td><td>Additional testing using separate login accounts is necessary to confirm that unauthorized operations are actually blocked.</td></tr>
<tr><td>Verification results</td><td>
After preparing the SQL scripts, they were intended to be executed in the PostgreSQL Query Tool through pgAdmin 4.

The scripts were organized and executed in the correct order to avoid permission-related errors.

First Verification: Database Inspection

The first script was executed to inspect the existing database structure.

The database name and schema were checked using PostgreSQL queries.

The following database information was identified through the inspection process:

<table>
<tr><th>Item</th><th>Result</th></tr>
<tr><td>Database Name</td><td>Actual PostgreSQL database name</td></tr>
<tr><td>Schema Name</td><td>public</td></tr>
<tr><td>Database Management System</td><td>PostgreSQL</td></tr>
<tr><td>Project</td><td>Restaurant Online Ordering and Table Reservation System</td></tr>
</table>
Expected result:

The database inspection queries should successfully display the existing schemas, tables, columns, sequences, views, and PostgreSQL
</td></tr>
</table>

	roles. **The actual database name and table structure should be recorded based on the query results.** **Second Verification: Role Creation** **The second script was executed to create the five group roles and their corresponding login accounts.** **A verification query was used to display the created roles from the PostgreSQL system catalog.** **Expected result:** **All five group roles and their corresponding login accounts should appear in the query results.** **The group roles should have the NOLOGIN attribute, while the corresponding user accounts should have the LOGIN attribute.** **Third Verification: Privilege Assignment** **The third script was executed to assign the required schema, table, and sequence privileges.** **The execution messages were checked to determine whether the GRANT statements completed successfully.** **Expected result:** **Each role should receive the permissions defined in the access-control plan.** **The administrator should receive full privileges on existing tables and sequences, while the manager, staff, customer, and read-only roles should receive SELECT privileges on existing tables.** **Fourth Verification: Privilege Restrictions** **The fourth script was executed to remove unnecessary privileges from restricted roles.** **Expected result:** **The staff role should not have DELETE privileges.**

	The customer role should not have INSERT privileges. **The customer role should not have UPDATE privileges.** **The customer role should not have DELETE privileges.** **The read-only role should not have INSERT privileges.** **The read-only role should not have UPDATE privileges.** **The read-only role should not have DELETE privileges.** **The read-only role should not have TRUNCATE privileges.** **The read-only role should not have USAGE or UPDATE privileges on sequences.** **Fifth Verification: Permission Checking** **The fifth script was executed to inspect the assigned permissions using PostgreSQL privilege-checking functions.** **The following results represent the expected permission configuration:**

Verification	Expected Result
Administrator can SELECT records	TRUE
Administrator can INSERT records	TRUE
Administrator can UPDATE records	TRUE
Administrator can DELETE records	TRUE

	Manager can SELECT records	**TRUE**
	Staff can SELECT records	**TRUE**
	Staff can DELETE records	**FALSE**
	Customer can SELECT records	**TRUE**
	Customer can INSERT records	**FALSE**
	Customer can UPDATE records	**FALSE**
	Customer can DELETE records	**FALSE**
	Read-only can SELECT records	**TRUE**
	Read-only can INSERT records	**FALSE**
	Read-only can UPDATE records	**FALSE**
	Read-only can DELETE records	**FALSE**

	The expected results demonstrate that the administrator has the intended database privileges, the manager and staff roles can view restaurant information, the customer role has restricted access, and the read-only role is limited to viewing records. **Note: These are expected results based on the prepared SQL scripts, not confirmed execution results. Actual results must be recorded after running the scripts and testing the permissions using the assigned PostgreSQL accounts.** --- ## Screenshots **The following screenshots should be included as supporting evidence of the implementation and verification process:** **Screenshot 1: G10_database_inspection.png** **Screenshot 2: G10_create_roles.png** **Screenshot 3: G10_grant_privileges.png** **Screenshot 4: G10_revoke_privileges.png** **Screenshot 5: G10_test_privileges.png** **The screenshots serve as supporting evidence of the database inspection, role creation, privilege assignment, privilege restrictions, and permission verification.** **Additional screenshots of the Restaurant Online Ordering and Table Reservation System database structure and PostgreSQL database setup may also be included to demonstrate that the existing database was successfully identified and used for the activity.**
Issues and fixes	**During the preparation and implementation of the access-control configuration, several possible issues were considered and addressed.** ### Database Name Mismatch **One possible error encountered during the preparation of the grant privileges script was:**

	ERROR: database "your_database_name" does not exist. **This happens when a placeholder database name is used and does not match the actual PostgreSQL database name.** **To resolve this issue, the current database name can be identified using the following query:** **SELECT current_database();** **The actual database name should then be used when preparing the GRANT CONNECT statement, if database-level CONNECT permissions are included.** ### Existing Database Roles **Running role creation commands repeatedly may cause errors if the roles already exist.** **To prevent this issue, the existing roles should be checked before executing the CREATE ROLE statements again.** **If the roles already exist, their current configuration should be reviewed before making changes.** **This helps avoid duplicate role creation errors when executing the SQL scripts.** ### Incorrect Execution Order **Granting privileges before creating the required roles can result in errors.** **To prevent this issue, the scripts were organized and executed in the following order:** **00_inspect_database.sql** **01_create_roles.sql** **02_grant_privileges.sql** **03_revoke_privileges.sql** **04_test_privileges.sql** **This ensures that the existing database structure is inspected first, the**

	required roles are created before permissions are assigned, and the final privileges are checked afterward. **Insufficient Database Permissions** **Some PostgreSQL commands require appropriate administrative privileges.** **The scripts should be executed using an authorized PostgreSQL administrator account or a database owner with sufficient privileges to ensure that role creation and privilege assignment commands can be completed successfully.** **Incorrect Table Names** **Granting privileges to tables that do not exist can result in relation errors.** **To avoid this issue, the grant privileges script uses:** **GRANT SELECT ON ALL TABLES IN SCHEMA public** **This allows PostgreSQL to apply the specified permissions to all existing tables in the public schema without manually entering individual table names.** **Similarly, the sequence privilege statements use:** **GRANT ALL PRIVILEGES ON ALL SEQUENCES IN SCHEMA public** **This allows permissions to be assigned to existing sequences in the specified schema.** **Database Connection and Schema Access** **Database-level CONNECT privileges do not automatically provide access to tables.** **The appropriate schema and table privileges must also be granted.** **To address this, the access-control configuration includes USAGE permission on the public schema and the required table and sequence privileges.** **If CONNECT permissions are required, they should also be granted separately on the actual database.**

	Future Database Objects **Newly created tables and sequences may not automatically receive the same permissions as existing objects.** **Default privileges should be configured for the correct object-owning role when necessary.** **This helps ensure that future database objects receive the intended access permissions.** **Customer Access Restrictions** **The customer role should not have unrestricted direct access to the restaurant database.** **Customers should normally interact with the system through the application's ordering and reservation features.** **The application should validate customer identity and restrict access to the customer's own records.** **This helps protect customer information and prevents unauthorized modifications to restaurant records.** **Application and Staff Permission Restrictions** **The manager and staff roles have different responsibilities from the administrator role.** **The initial configuration provides SELECT access to existing tables for these roles.** **Additional INSERT and UPDATE privileges should be granted only to the specific tables required for their responsibilities.** **This helps reduce unnecessary database access and improves the implementation of the Principle of Least Privilege.** **PostgreSQL Privilege Inheritance** **PostgreSQL allows users to inherit permissions through role memberships and other grants.** **Revoking a privilege from one group role does not necessarily remove the same privilege if the user receives it through another role.**

	To resolve this issue, role memberships and effective privileges should be reviewed during testing.
Feedback to owners	**The Restaurant Online Ordering and Table Reservation System provides functionality for managing online food orders and table reservations. Implementing Role-Based Access Control can further improve the security and organization of database operations.** **The following recommendations are provided to the project owners:** **1. Maintain separate database roles for administrators, managers, staff, customers, and read-only users.** **2. Avoid using administrator-level database credentials for regular application activities.** **3. Assign actual login accounts to the appropriate group roles.** **4. Use strong passwords and protect database credentials.** **5. Review database permissions whenever new tables and application features are added.** **6. Restrict staff access to the specific order and reservation tables required for their responsibilities.** **7. Restrict customer access to their own information and transactions.** **8. Use read-only accounts for users who only need to view database information.** **9. Apply appropriate database privileges to newly created tables and sequences.** **10. Perform regular database backups to help prevent data loss.** **11. Avoid granting unnecessary administrative privileges to application accounts.** **12. Regularly test database access restrictions to ensure that unauthorized operations are prevented.** **13. Keep database credentials and sensitive configuration files out of public GitHub repositories.** **14. Review database roles and privileges regularly to prevent unnecessary access.** **15. Implement appropriate application-level validation to protect customer records and reservation information.** **These recommendations can help improve the security, reliability, and maintainability of the Restaurant Online Ordering and Table Reservation System.**
AI usage declaration	**Artificial Intelligence (AI) tools were used as a supplementary resource**

	during the activity to assist in understanding PostgreSQL role creation, database privileges, SQL syntax, and documentation preparation. **AI assistance was used to help organize the SQL scripts, explain the purpose of each access-control role, prepare the database access-control plan, and identify possible implementation issues and their corresponding solutions.** **The generated SQL commands and documentation were reviewed and prepared based on the available Restaurant Online Ordering and Table Reservation System project information.** **The SQL scripts must be validated through PostgreSQL execution using pgAdmin 4. The actual database name, tables, and assigned privileges must be checked before final submission.** **The student remains responsible for checking the correctness of the implementation, understanding the assigned permissions, and verifying the actual execution results.** **All passwords and other sensitive credentials must be excluded from the submitted repository, screenshots, and documentation.**